

## **ЗВІТ**

### **до лабораторної роботи №2**

з дисципліни «Функціональне програмування»

Тема: «Функції вищого порядку, lambda, композиція»

Варіант 1 — «Клієнти інтернет-магазину»

Виконав(ла): \_\_\_\_\_ Бардак Дмитро \_\_\_\_\_

Група: \_\_\_\_\_ ФсП-21 \_\_\_\_\_

Перевірів(ла): \_\_\_\_\_

## 1. Мета роботи

Набути практичних навичок використання функцій вищого порядку, lambda-виразів, функціональної композиції, операцій filter/map/reduce та побудови конвеєра перетворень над списком записів без мутації вхідних даних.

## 2. Постановка задачі

Опрацювати список клієнтів інтернет-магазину з полями id, name, age, city, purchases. Відфільтрувати повнолітніх, обчислити суму покупок, підсилити суму для заданого міста, відсортувати за total спадно з тай-брейкером id зростаюче та сформувані Top-N. Окремо обчислити count, sum\_total та avg\_total за допомогою reduce.

## 3. Структура програми

- core.py — Record, чисті трансформації, compose, конвеєр та reduce-агрегація.
- main.py — читання JSON, параметри CLI, запуск і виведення результатів.
- sample\_data.json — приклад вхідних даних.
- tests/test\_core.py — тести Top-N, відсутності мутацій, агрегації та композиції.
- README.md — інструкція запуску.
- requirements.txt, pyproject.toml — залежності та конфігурація.

## 4. Алгоритм виконання

1. Нормалізувати імена клієнтів у Title Case.
2. Залишити лише записи з age  $\geq$  18.
3. Додати total = sum(purchases).
4. Для заданого міста помножити total на factor.
5. Відсортувати за (-total, id).
6. Вибрати перші N записів.
7. Через reduce обчислити count, sum\_total, avg\_total.

## 5. Основні фрагменти коду

**TypedDict Record**

```
class Record(TypedDict, total=False):
    id: int
    name: str
    age: int
    city: str
    purchases: list[float]
    total: float
```

## Композиція

```
def compose(*funcs: Callable):
    def _composed(value):
        for func in reversed(funcs):
            value = func(value)
        return value
    return _composed
```

## Конвеєр

```
return compose(
    take(top_n),
    sort_by_total_desc(),
    boost_city(city, factor),
    with_total(),
    only_adults(18),
    normalize_names(),
)
```

## 6. Результат виконання

Для sample\_data.json, Top-3, city=Delhi, factor=1.1 отримуємо:

```
4: Peter White, Delhi, total=220.00
3: Maria Green, London, total=200.00
5: Olivia Stone, Paris, total=170.00
```

```
count = 3
sum_total = 590.00
avg_total = 196.67
```

## 7. Тестування

Перевіряються Top-N і порядок сортування, незмінність початкових даних, reduce-агрегація та compose.

4 passed in 0.04s

## 8. Висновок

Реалізовано функціональний конвеєр для клієнтів інтернет-магазину без мутації вхідних даних. Використано функції вищого порядку, `lambda`, генераторні аналоги `filter/map`, композицію, сортування за двома ключами, `Top-N` та `reduce`-агрегацію. I/O відокремлено від чистої логіки, а основні вимоги перевіряються `pytest`-тестами.